

TRAVAUX PRATIQUES Compilation

TP 1 — Analyse Lexicale avec Flex

Réalisée par : El Alaoui Hanane

Groupe de TP : 1

Code apogée : 23065496

Encadrée par : Prof Mohammed AL ACHHAB

Commandes de compilation:

Depuis le dossier tpthl, j'exécute dans l'invite de commandes :

```
C:\Users\admin>cd C:\Tpthl
```

```
C:\Tpthl>win_flex TP1_EX1.1
```

→ Génère le fichier *lex.yy.c*

```
C:\Tpthl>gcc lex.yy.c -o ex1.exe
```

→ Génère l'exécutable *ex1.exe*

```
C:\Tpthl>ex1.exe
Entrez du texte (Ctrl+D pour terminer) :
```

→ Lance l'analyseur lexical

Exercice1 : Reconnaissance de nombres binaires (clavier)

✓ Points clés

- Le motif `[01]+` reconnaît 1 ou plusieurs chiffres binaires.
- `yytext` contient le lexème reconnu (chaîne courante).
- `yywrap()` retourne 1 pour signaler la fin du fichier.
- Le point `'.'` capture tout autre caractère (ignoré ici).

Le code flex :

```
*TP1_EX1.1 - Bloc-notes
Fichier Edition Format Affichage Aide

%{
#include <stdio.h>
%}

%%
[01]+ { printf("Nombre binaire : %s\n", yytext); }
\n    { /* ignorer les nouvelles lignes */ }
.      { /* ignorer les autres caracteres */ }
%%

int yywrap() { return 1; }

int main() {
    printf("Entrez du texte (Ctrl+D pour terminer) :\n");
    yylex();
    return 0;
}
```

Le lancement et le teste :

```
C:\Tpthl>ex1.exe
Entrez du texte (Ctrl+D pour terminer) :
1100
Nombre binaire : 1100
344
00
Nombre binaire : 00
990
Nombre binaire : 0
```

Exercice2 : Lecture depuis un fichier input.txt

Le code flex :

```
TP1_EX2.1 - Bloc-notes
Fichier Edition Format Affichage Aide
%{
#include <stdio.h>
}%

%%

[01]+ { printf("Nombre binaire : %s\n", yytext); }
.      { /* ignorer */ }
\n     { /* ignorer */ }
%%

int yywrap() { return 1; }

int main() {
extern FILE *yyin;
yyin = fopen("input.txt", "r");
if (!yyin) {
fprintf(stderr, "Erreur : impossible d'ouvrir input.txt\n");
return 1;
}
yylex();
fclose(yyin);
return 0;
}
```

```
input.txt - Bloc-notes
Fichier Edition Format Affichage Aide
Hello 1110 world 0100 333
```

Le lancement et le teste :

```
C:\Tpthl>win_flex TP1_EX2.1
C:\Tpthl>gcc lex.yy.c -o ex2.exe
C:\Tpthl>ex2.exe
Nombre binaire : 1110
Nombre binaire : 0100
```

Exercice3 : Comptage de voyelles, consonnes et ponctuation

Expressions régulières utilisées

Catégorie	Motif Flex
Voyelles	[aeiouAEIOU]
Consonnes	
Ponctuation	

Le code flex :

```
TP1_EX3.I - Bloc-notes
Fichier Edition Format Affichage Aide
%{
#include <stdio.h>
int voyelles = 0, consonnes = 0, ponctuation = 0;
}%

%%
[aeiouAEIOU]          { voyelles++; }
[bcdfghjklmnpqrstvwxyzBCDFGHJKLMPQRSTUVWXYZ] { consonnes++; }
[.,;:!?\\-\\'\\\"\\(\\)\\[\\]\\{\\}\\}"] { ponctuation++; }
.                      { /* autres caracteres */ }
\\n                    { /* nouvelles lignes */ }
%%

int yywrap() { return 1; }

int main() {
    extern FILE *yyin;
    yyin = fopen("texte.txt", "r");
    if (!yyin) {
        fprintf(stderr, "Erreur : impossible d'ouvrir texte.txt\\n");
        return 1;
    }
    yylex();
    printf("Voyelles      : %d\\n", voyelles);
    printf("Consonnes     : %d\\n", consonnes);
    printf("Ponctuations  : %d\\n", ponctuation);
    fclose(yyin);
    return 0;
}
```

```
texte.txt - Bloc-notes
Fichier Edition Format Affichage Aide
Bonjour, je m'appelle Hanane.
Le langage Flex est utile pour l'analyse lexicale.
Ce fichier est utilisé pour tester l'exercice 3 du tp 1.
Les consonnes, voyelles et ponctuations sont comptées ici.
```

Le lancement et le teste :

```
C:\Tp1>gcc lex.yy.c -o ex3.exe
C:\Tp1>ex3.exe
Voyelles      : 64
Consonnes     : 90
Ponctuations  : 9
```

Exercice4 : Analyseur Lexical Complet

Conception d'un analyseur lexical pour un mini-langage avec les unités lexicales suivantes :

Unité Lexicale	Description / Exemples
Identificateurs	Lettre suivi de lettres, chiffres, _ ex: monVar, x1, ma_var
Mots réservés	debut, fin, si, alors, sinon, finsi
Chaînes	Entre guillemets ex: "bonjour monde"
Entiers décimaux	0, 42, 1234
Opérateurs	+ - * / mod ++ -- < > == !=
Affectation	:=

Partie 1 : Affichage des unités lexicales

Le code flex :

```
*TP1_EX4_P1.I - Bloc-notes
Fichier Edition Format Affichage Aide

%{
#include <stdio.h>
%}
LETTRE    [a-zA-Z]
CHIFFRE    [0-9]
ID        {LETTRE}({LETTRE}|{CHIFFRE}|_)*
ENTIER     {CHIFFRE}+
CHAINE     \"[^\"]*\"
%%
"debut"    { printf("Mot reserve : %s\n", yytext); }
"fin"      { printf("Mot reserve : %s\n", yytext); }
"si"       { printf("Mot reserve : %s\n", yytext); }
"alors"    { printf("Mot reserve : %s\n", yytext); }
"sinon"    { printf("Mot reserve : %s\n", yytext); }
"finsi"    { printf("Mot reserve : %s\n", yytext); }
"+"|"-"|"*"|"/" { printf("Operateur : %s\n", yytext); }
"mod"      { printf("Operateur : %s\n", yytext); }
"++"|"--"  { printf("Operateur : %s\n", yytext); }
"<"|>"|="|"!=" { printf("Operateur : %s\n", yytext); }
":="       { printf("Affectation : %s\n", yytext); }
{CHAINE}   { printf("Chaine : %s\n", yytext); }
{ENTIER}   { printf("Entier : %s\n", yytext); }
{ID}       { printf("Identif. : %s\n", yytext); }
[ \t\n]    { /* ignorer espaces */ }
.          { printf("ERREUR : caractere inconnu '%s'\n", yytext); }
%%
int yywrap() { return 1; }
int main() {
    extern FILE *yyin;
    yyin = fopen("source.txt", "r");
    if (!yyin) { fprintf(stderr, "Impossible d'ouvrir source.txt\n"); return 1; }
    yylex();
    fclose(yyin);
    return 0; }
}
```

```
source.txt - Bloc-notes
Fichier Edition Format Affichage Aide

    si x > 5 alors
        x := x + 1
    fin
fin
```

Le lancement et le teste :

```
C:\Tpth1>win_flex TP1_EX4_P1.1
C:\Tpth1>gcc lex.yy.c -o ex4.P1.exe
C:\Tpth1>ex4.P1.exe
Mot reserve : debut
Identif. : x
Affectation : :=
Entier : 10
Mot reserve : si
Identif. : x
Operateur : >
Entier : 5
Mot reserve : alors
Identif. : x
Affectation : :=
Identif. : x
Operateur : +
Entier : 1
Mot reserve : fin
Mot reserve : fin
```

Partie 2 : Génération HTML colorisée

Modification pour générer un fichier output.html avec la mise en forme suivante :

Catégorie	Style HTML appliqué
Mots réservés	mot → Gras
Identificateurs	id → Bleu
Chaînes de caractères	chaîne → Italiques
Opérateurs	op → Rouge

Le code flex :

```
*TP1_EX4_P2.I - Bloc-notes
Fichier Edition Format Affichage Aide

%{
#include <stdio.h>
FILE *html_out;
}%
LETTRE    [a-zA-Z]
CHIFFRE   [0-9]
ID        {LETTRE}{LETTRE}|{CHIFFRE}|_)*
ENTIER    {CHIFFRE}+
CHAINE    \"[^\"]*\"
%%
"debut"|"fin"|"si"|"alors"|"sinon"|"finsi" {
    fprintf(html_out, "<strong>%s</strong>", yytext);
}
"+"|"-"|"*"|"/"|"mod"|"++"|"--"|"<"|">"|"=="|"!="|":"|":"=" {
    fprintf(html_out, "<span style=\"color:red\">%s</span>", yytext);
}
{CHAINE} { fprintf(html_out, "<em>%s</em>", yytext); }
{ENTIER} { fprintf(html_out, "%s", yytext); }
{ID}     { fprintf(html_out, "<span style=\"color:blue\">%s</span>", yytext); }
\n      { fprintf(html_out, "<br>\n"); }
[ \t]   { fprintf(html_out, "%s", yytext); }
.       { fprintf(html_out, "%s", yytext); }
%%
int yywrap() { return 1; }
int main() {
    extern FILE *yyin;
    yyin = fopen("source.txt", "r");
    html_out = fopen("output.html", "w");
    fprintf(html_out, "<!DOCTYPE html>\n<html>\n<body>\n<pre>\n");
    yylex();
    fprintf(html_out, "</pre>\n</body>\n</html>");
    fclose(yyin); fclose(html_out);
    printf("Fichier output.html genere.\n");
    return 0; }
```

```
source.txt - Bloc-notes
Fichier Edition Format Affichage Aide
debut
    x := 10
    y := 25
    resultat := x + y

    si x > 5 alors
        compteur := compteur + 1
        message := "valeur correcte"
    sinon
        compteur := compteur - 1
        message := "valeur incorrecte"
    fin

    total := resultat * 2
    reste := total mod 3
    x++
    y--
fin
```

Le lancement et le teste :

```
C:\Tpth1>win_flex TP1_EX4_P2.1
C:\Tpth1>gcc lex.yy.c -o ex4.P2.exe
C:\Tpth1>win_flex TP1_EX4_P2.1
C:\Tpth1>gcc lex.yy.c -o ex4_P2.exe
C:\Tpth1>ex4_P2.exe
Fichier output.html genere.
```

```
debut

x := 10
y := 25

resultat := x + y

si x > 5 alors
    compteur := compteur + 1
    message := "valeur correcte"
sinon
    compteur := compteur - 1
    message := "valeur incorrecte"
finsi

total := resultat * 2
reste := total mod 3
x++
y--

fin
```

Partie 3: Erreurs lexicales et Table des Symboles

⚠ Gestion des erreurs

La règle catchall `. { ... }` en dernière position intercepte tout caractère non reconnu. Flex applique la règle la plus longue en premier, donc le `.` ne sera atteint que si aucune autre règle ne correspond.

Le code flex :

source.txt - Bloc-notes

```
debut
    x := 10
    y@ := 25
    si x > 5 alors
        x := x + 1
    finsi
    z := x # y
fin
```

Le lancement et le teste :


```

C:\Tpth1>win_flex TP1_EX4_P3.1

C:\Tpth1>gcc lex.yy.c -o ex4.P3.exe

C:\Tpth1>win_flex TP1_EX4_P3.1

C:\Tpth1>gcc lex.yy.c -o ex4_P3.exe

C:\Tpth1>ex4.P3.exe
Mot reserve : debut
Identif.    : x
Operateur   : :=
Entier      : 10
Identif.    : y
ERREUR LEXICALE : caractere illegal '@' (ligne 1)
Operateur   : :=
Entier      : 25
Mot reserve : si
Identif.    : x
Operateur   : >
Entier      : 5
Mot reserve : alors
Identif.    : x
Operateur   : :=
Identif.    : x
Operateur   : +
Entier      : 1
Mot reserve : fin
Identif.    : z
Operateur   : :=
Identif.    : x
ERREUR LEXICALE : caractere illegal '#' (ligne 1)
Identif.    : y
Mot reserve : fin

=== TABLE DES SYMBOLES ===
Identifiant      Occurrences
-----
x                 5
y                 2
z                 1

=== BILAN ===
Identifiants uniques : 3
Erreurs lexicales    : 2

```